

***Multi – level Linked List – Modified Insert After Element
(Main List Insertion) – Program Analysis.***

Program:

```
Node *searchNode(Node *current, int target)
{
    if (current == nullptr)
        return nullptr;

    if (current->data == target)
        return current;

    Node *foundInChild = searchNode(current->child, target);
    if (foundInChild != nullptr)
    {
        return foundInChild;
    }
    return searchNode(current->next, target);
}

void insertAfterElement()
{
    if (head == nullptr)
    {
        cout << "List is empty. Please insert an element first." << endl;
        return;
    }

    int target, data, type;
    cout << "Enter the element to search for: ";
    cin >> target;

    Node *temp = searchNode(head, target);

    if (temp == nullptr)
    {
```

```

        cout << "Element " << target << " not found anywhere
in the list." << endl;
        return;
    }

    cout << "Enter data to insert: ";
    cin >> data;
    Node *newNode = createNode(data);

    cout << "Insert as (1) Next Node or (2) Child Node? ";
    cin >> type;

    if (type == 1)
    {
        newNode->next = temp->next;
        temp->next = newNode;
        cout << "Inserted " << data << " as NEXT of " <<
target << endl;
    }
    else if (type == 2)
    {
        newNode->next = temp->child;
        temp->child = newNode;
        cout << "Inserted " << data << " as CHILD of " <<
target << endl;
    }
    else
    {
        cout << "Invalid option." << endl;
        free(newNode);
    }
}

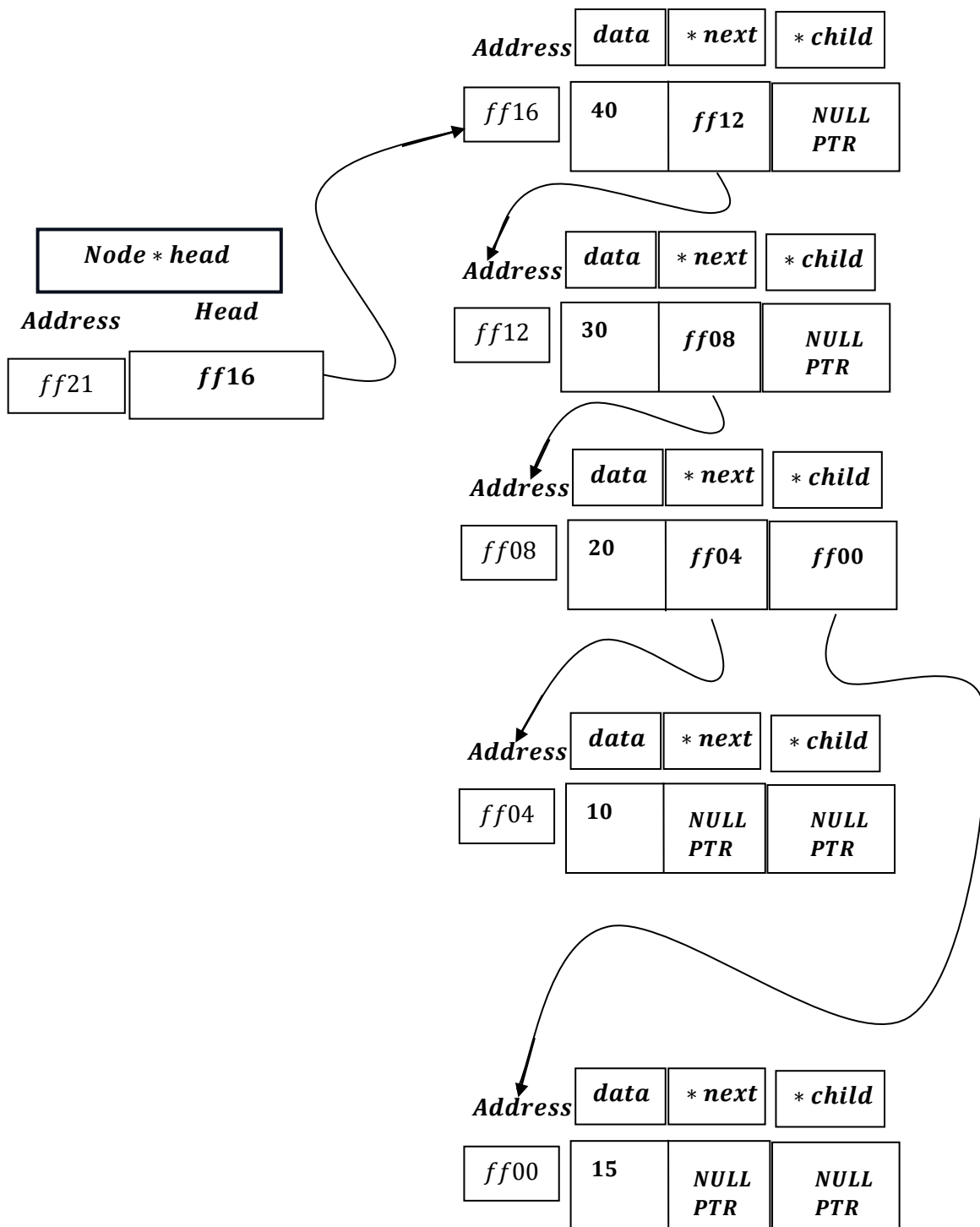
int main()
{
    int choice;
    while (true)
    {

```

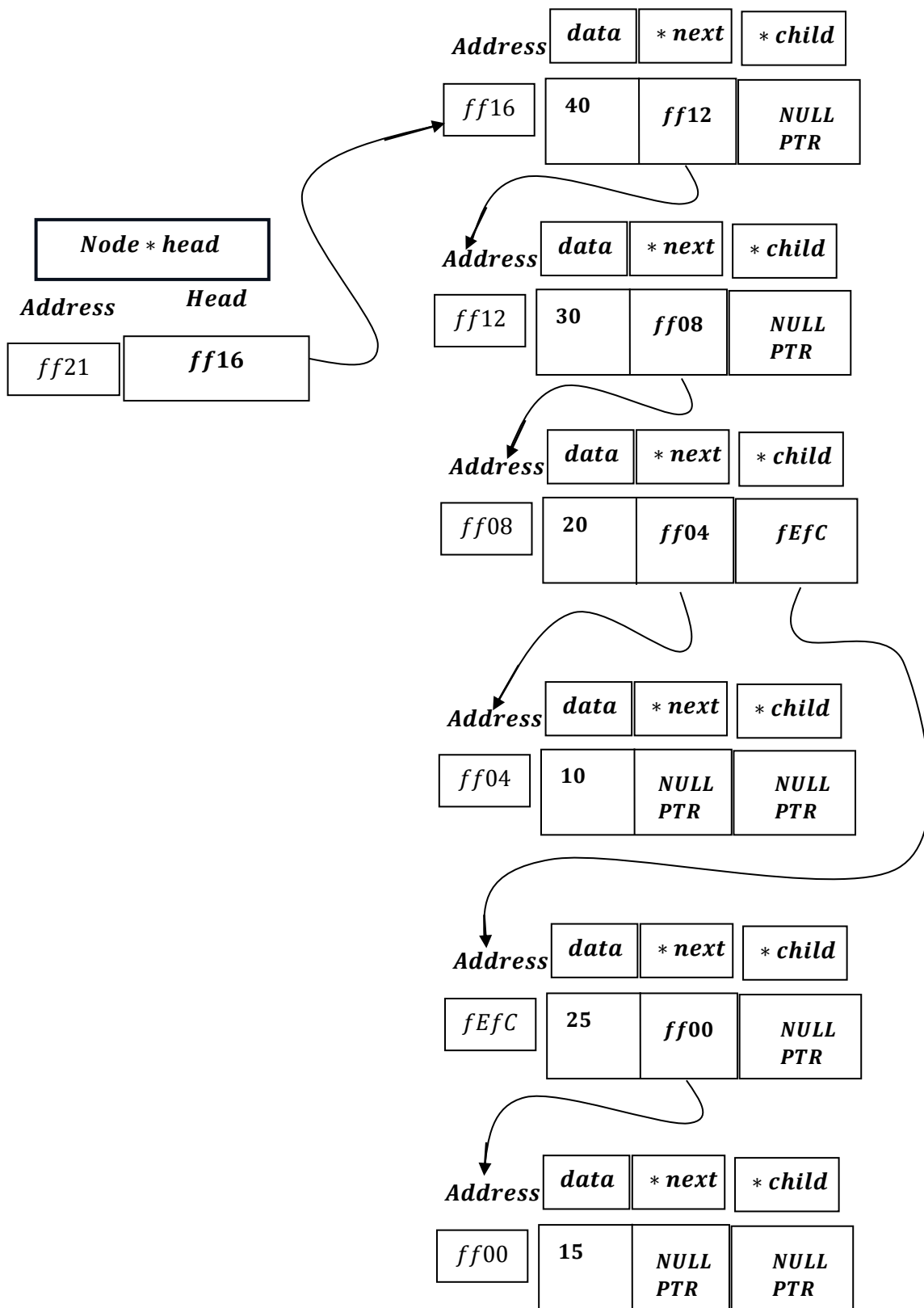
```
        cout << "4. Insert After Element (Link or Child)" << endl;
    switch (choice)
    {
    case 4:
        insertAfterElement();
    default:
        cout << "Invalid choice" << endl;
    }
}
return 0;
}
```

Program Analysis:

We had earlier :



Next again if we insert child in address: ff08, we get:



Lets say we want to enter, after address ff04 .

And we want to enter data := 5.

```
cout << "Enter the element to search for: ";  
cin >> target;
```

target: 10

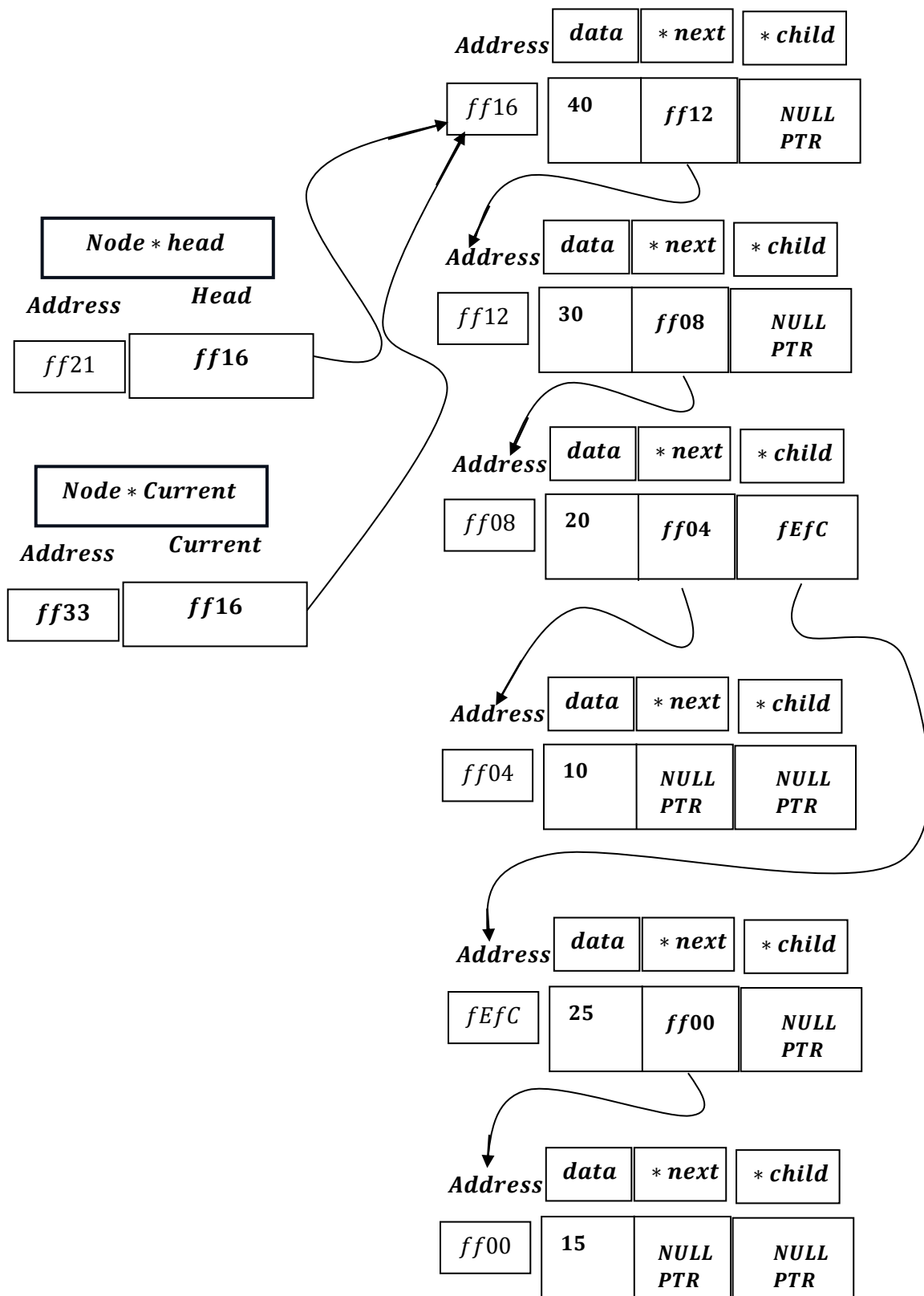
```
Node *temp = searchNode(head, target);
```

*∴ Node * temp = searchNode(head: ff16, target: 10) ;*

Now,

```
Node *searchNode(Node *current, int target)
```

∴ searchNode(Current = ff16, target: 10);



```
if (current == nullptr)
    return nullptr;
```

Current $\neq$ *NULLPTR*

```
if (current->data == target)
    return current;
```

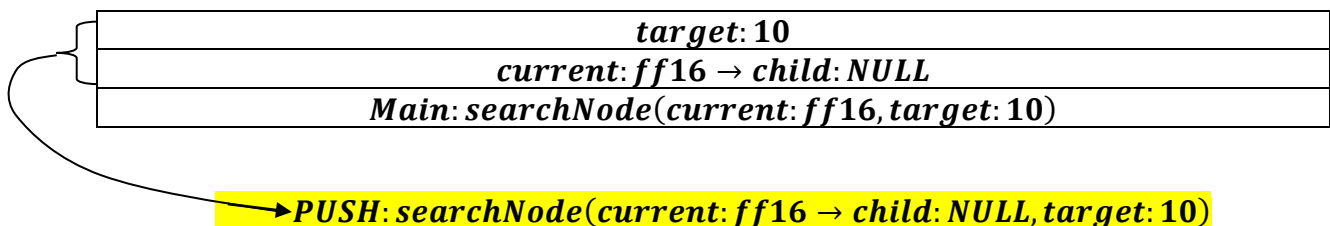
current $\rightarrow$ *data*: 40 $\neq$ *target*: 10

```
Node *foundInChild = searchNode(current->child, target);
```

current $\rightarrow$ *child* = *NULL* and *target* = 10.

$\therefore$ *Node* * *foundInChild*
= *searchNode*(*current*: ff16 $\rightarrow$ *child*: *NULL*, *target*: 10);

Recursion child – 1 (Push)



```
Node *searchNode(Node *current, int target)
```

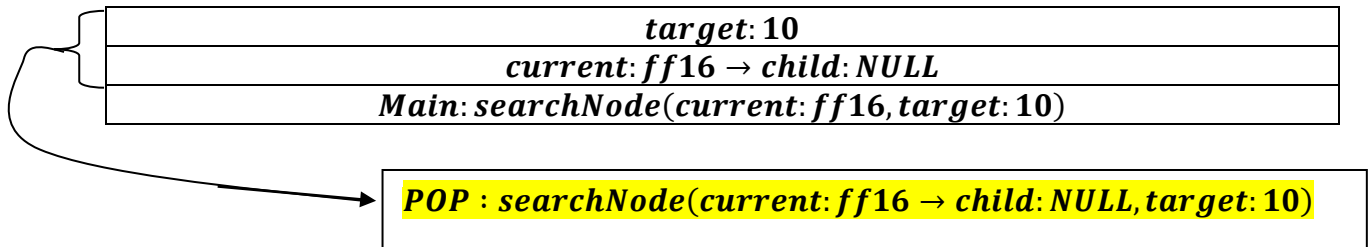
Assigning : *Node* * *searchNode*(*Node* * *current* = *NULL*, *int* *target* = 10)

```
if (current == nullptr)
    return nullptr;
```


Current = NULLPTR hence return nullptr;

As soon as it returns NULLPTR , then:

Recursion child – 1 (Pop)



As soon as it pop, NULLPTR, now :

```
Node *foundInChild = searchNode(current->child, target);
```

`Node * foundInChild = NULLPTR.`

```
if (foundInChild != nullptr)
{
    return foundInChild;
}
```

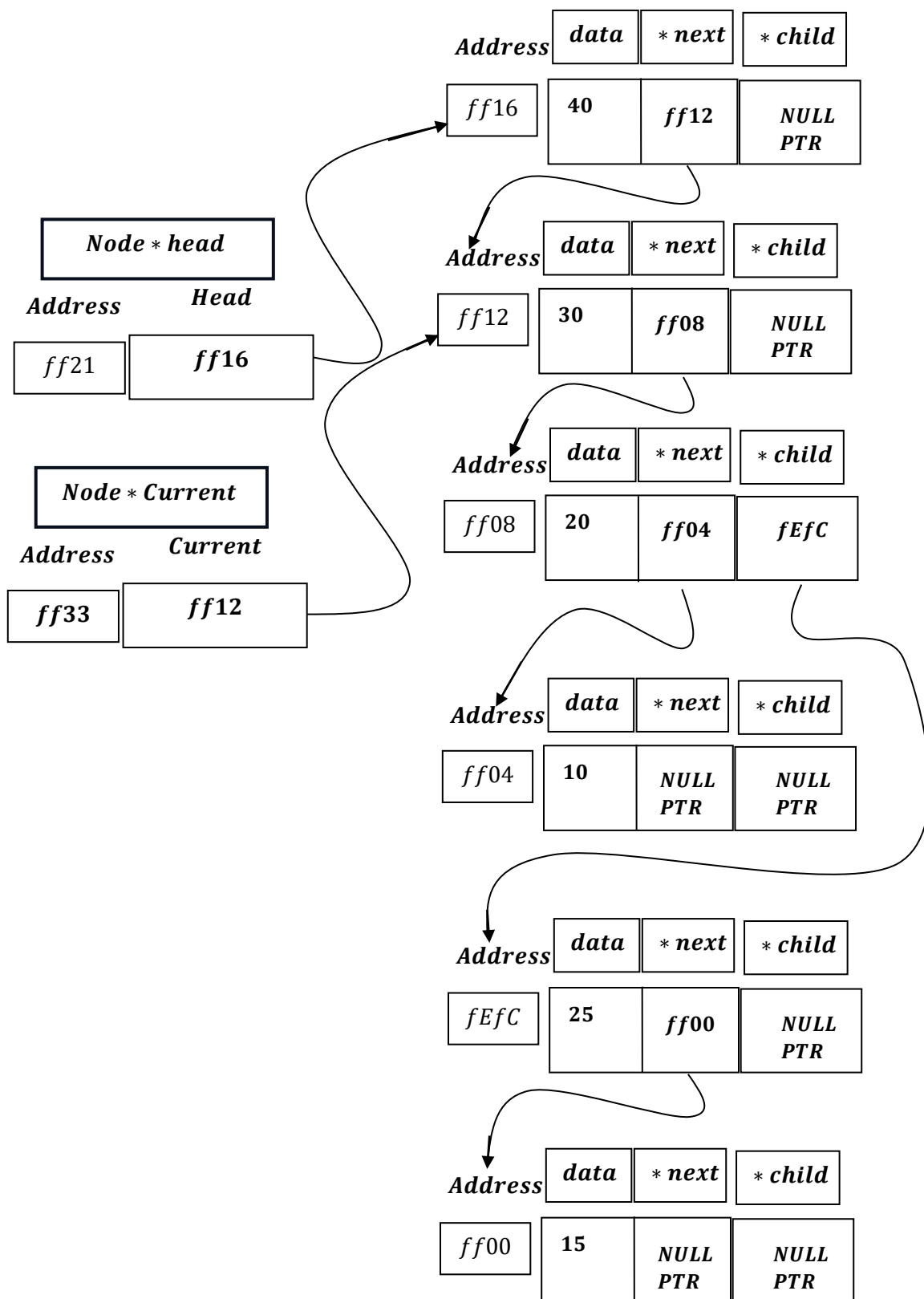
*But **`foundInChild = NULLPTR;`** hence condition is false.*

```
return searchNode(current->next, target);
```

`searchNode(current: ff16 -> next: ff12, target: 10);`

Recursion next push – 1

<code>searchNode(current: ff12, target: 10)</code>
<code>Main: searchNode(current: ff16, target: 10)</code>



```
if (current == nullptr)
    return nullptr;
```

Current $\neq$ *NULLPTR*

```
if (current->data == target)
    return current;
```

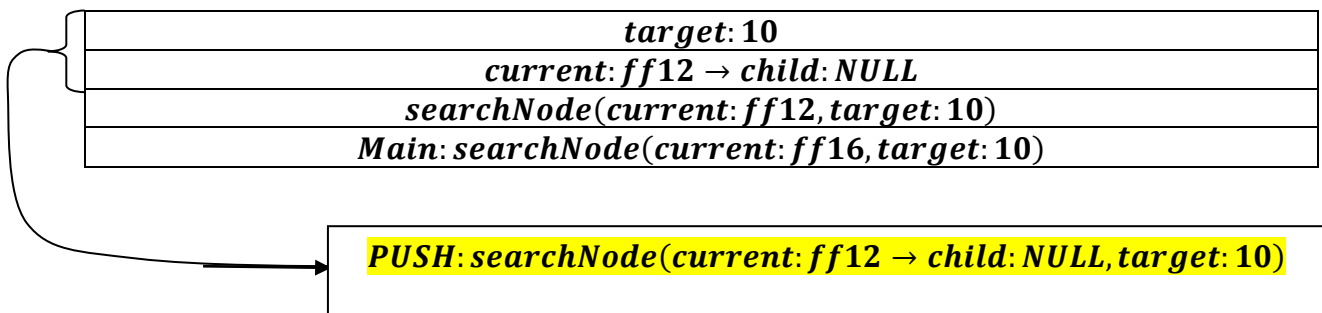
current $\rightarrow$ *data*: 30 $\neq$ *target*: 10

```
Node *foundInChild = searchNode(current->child, target);
```

current: ff12 $\rightarrow$ *child* = *NULL* and *target* = 10.

$\therefore$ *Node* * *foundInChild*
 $=$ *searchNode*(*current*: ff12 $\rightarrow$ *child*: *NULL*, *target*: 10);

Recursion child – 2 (Push)



```
Node *searchNode(Node *current, int target)
```

Assigning : *Node* * *searchNode*(*Node* * *current* = *NULL*, *int* *target* = 10)

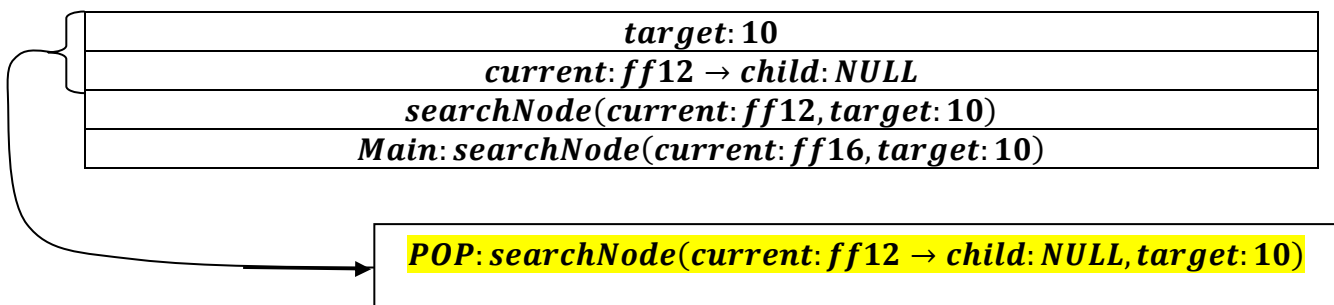
```
if (current == nullptr)
```

```
return nullptr;
```

Current = NULLPTR hence return nullptr;

As soon as it returns NULLPTR , then:

Recursion child – 2 (POP)



As soon as it pop, NULLPTR, now :

```
Node *foundInChild = searchNode(current->child, target);
```

Node * foundInChild = NULLPTR.

```
if (foundInChild != nullptr)
{
    return foundInChild;
}
```

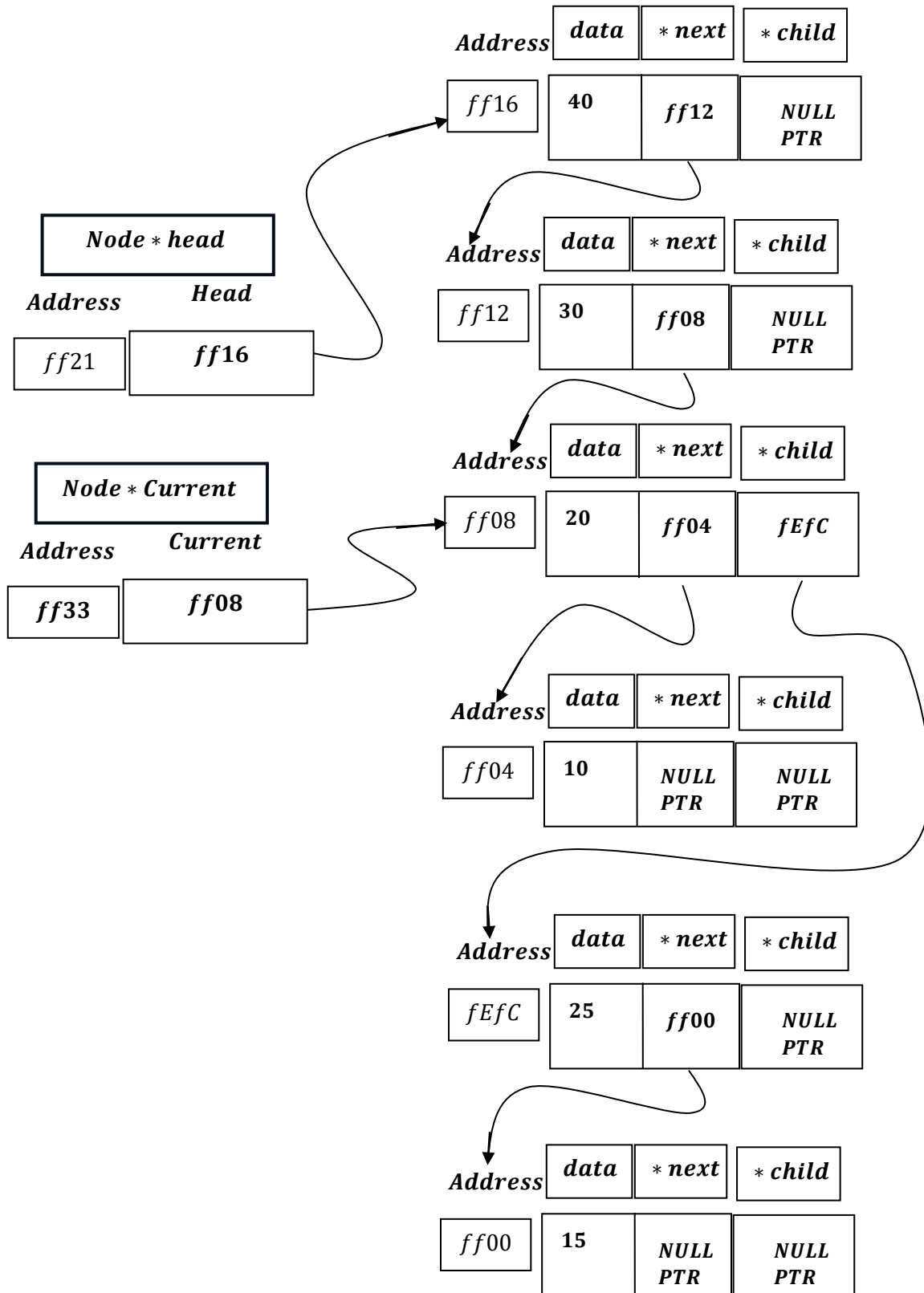
But foundInChild = NULLPTR; hence condition is false.

```
return searchNode(current->next, target);
```

searchNode(current: ff12 → next: ff08, target: 10);

Recursion next push – 2

<i>searchNode(current: ff08, target: 10)</i>
<i>searchNode(current: ff12, target: 10)</i>
<i>Main: searchNode(current: ff16, target: 10)</i>



```
if (current == nullptr)
    return nullptr;
```

Current $\neq$ *NULLPTR*

```
if (current->data == target)
    return current;
```

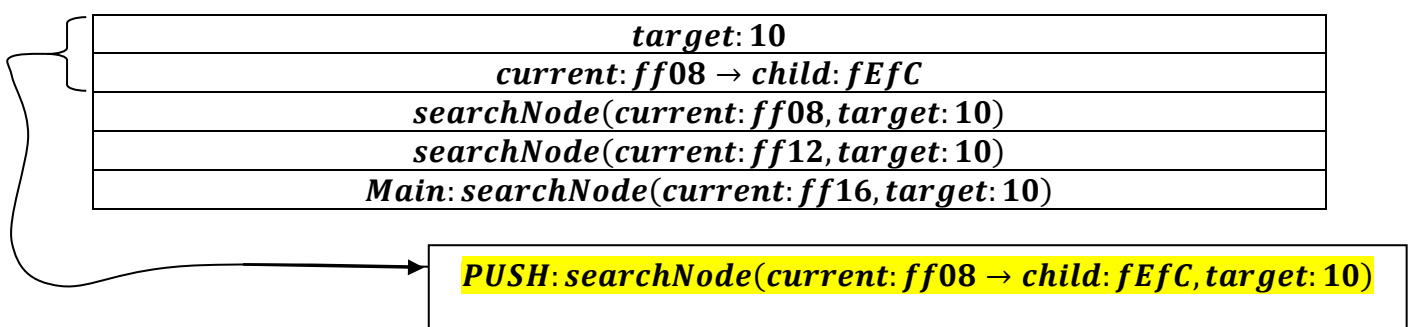
current $\rightarrow$ *data*: 20 $\neq$ *target*: 10

```
Node *foundInChild = searchNode(current->child, target);
```

current $\rightarrow$ *child* = *fEfC* and *target* = 10.

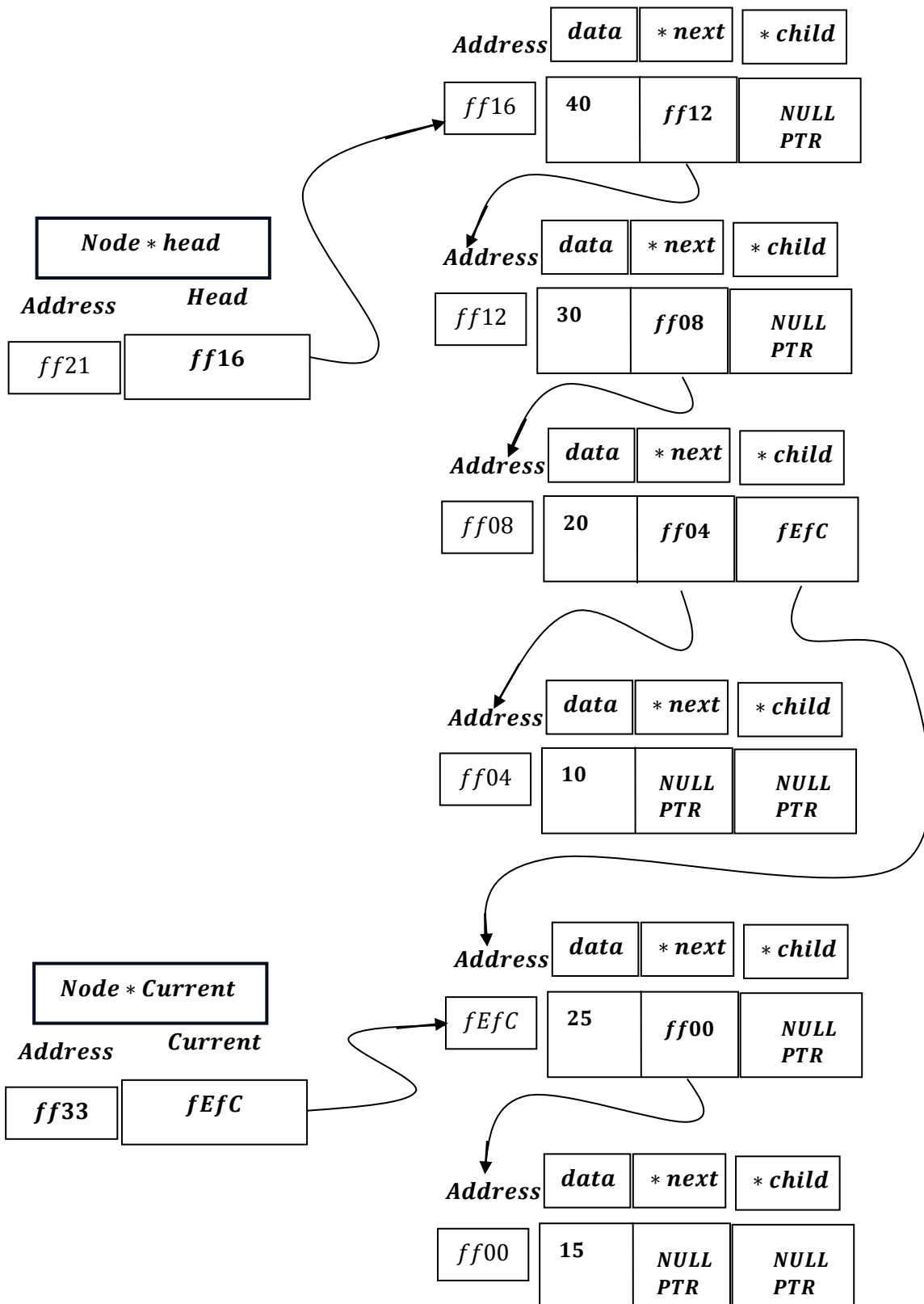
$\therefore$ *Node* * *foundInChild*
 $=$ *searchNode*(*current*: *ff08* $\rightarrow$ *child*: *fEfC*, *target*: 10);

Recursion child push – 3



```
Node *searchNode(Node *current, int target)
```

*Node * searchNode(Node * current = fEfC ,target = 10)*



```
if (current == nullptr)
    return nullptr;
```

current $\neq$ *nullptr* , hence condition is false.

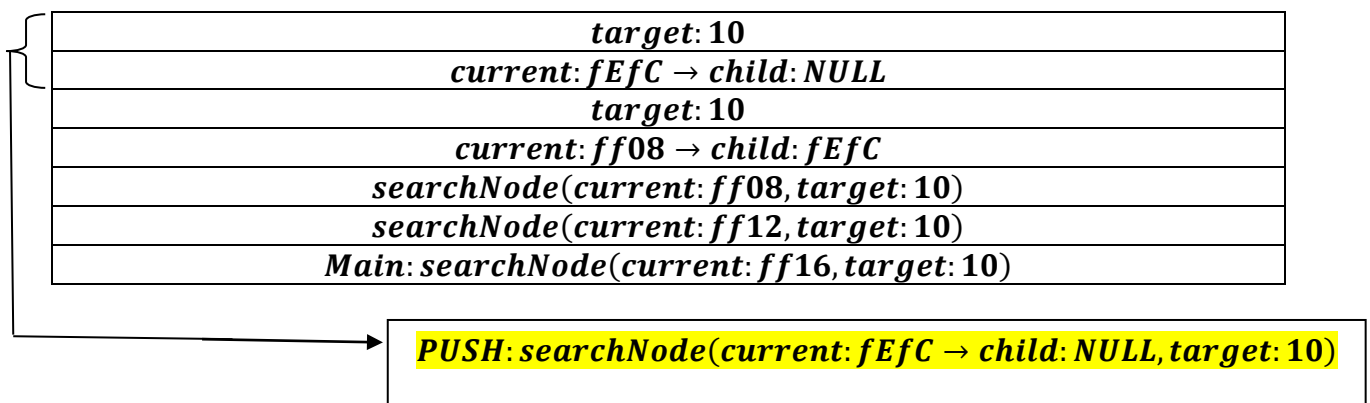
```
if (current->data == target)
    return current;
```

current $\rightarrow$ *data* : 25 $\neq$ *target* : 10 , hence condition fails.

```
Node *foundInChild = searchNode(current->child, target);
```

searchNode(*current*: *fEfC* $\rightarrow$ *child*: *NULL* , *target*: 10)

Recursion child push – 4



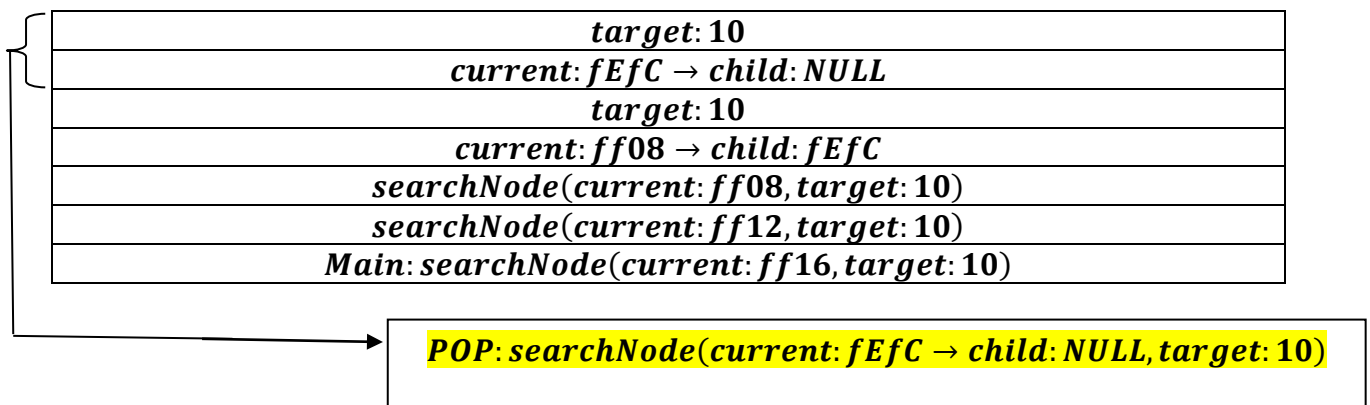

```
Node *searchNode(Node *current, int target)
```

*Node * searchNode(Node * current = NULL ,target = 10)*

```
if (current == nullptr)
    return nullptr;
```

current = nullptr , hence condition is true, return nullptr.

Recursion child pop – 4



As soon as it pop, NULLPTR, now :

```
Node *foundInChild = searchNode(current->child, target);
```

Node * foundInChild = NULLPTR.

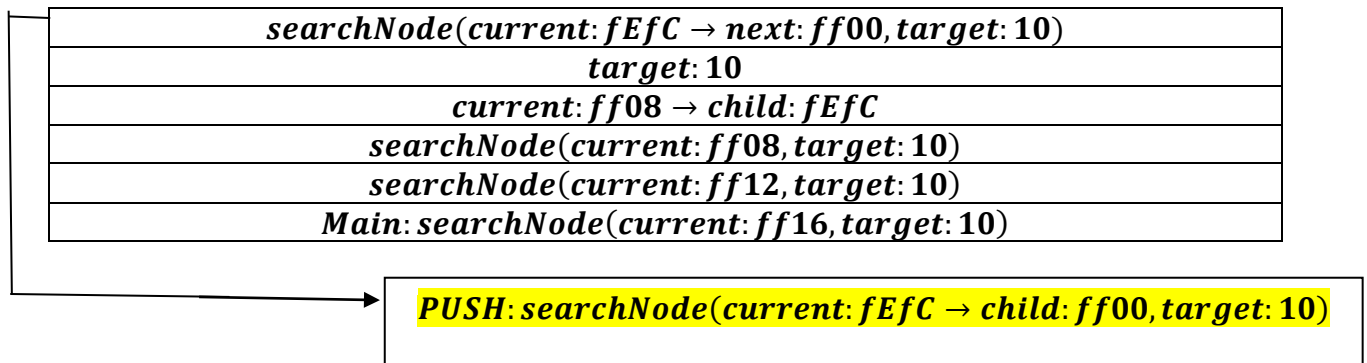
```
if (foundInChild != nullptr)
{
    return foundInChild;
}
```

But foundInChild = NULLPTR; hence condition is false.

```
return searchNode(current->next, target);
```

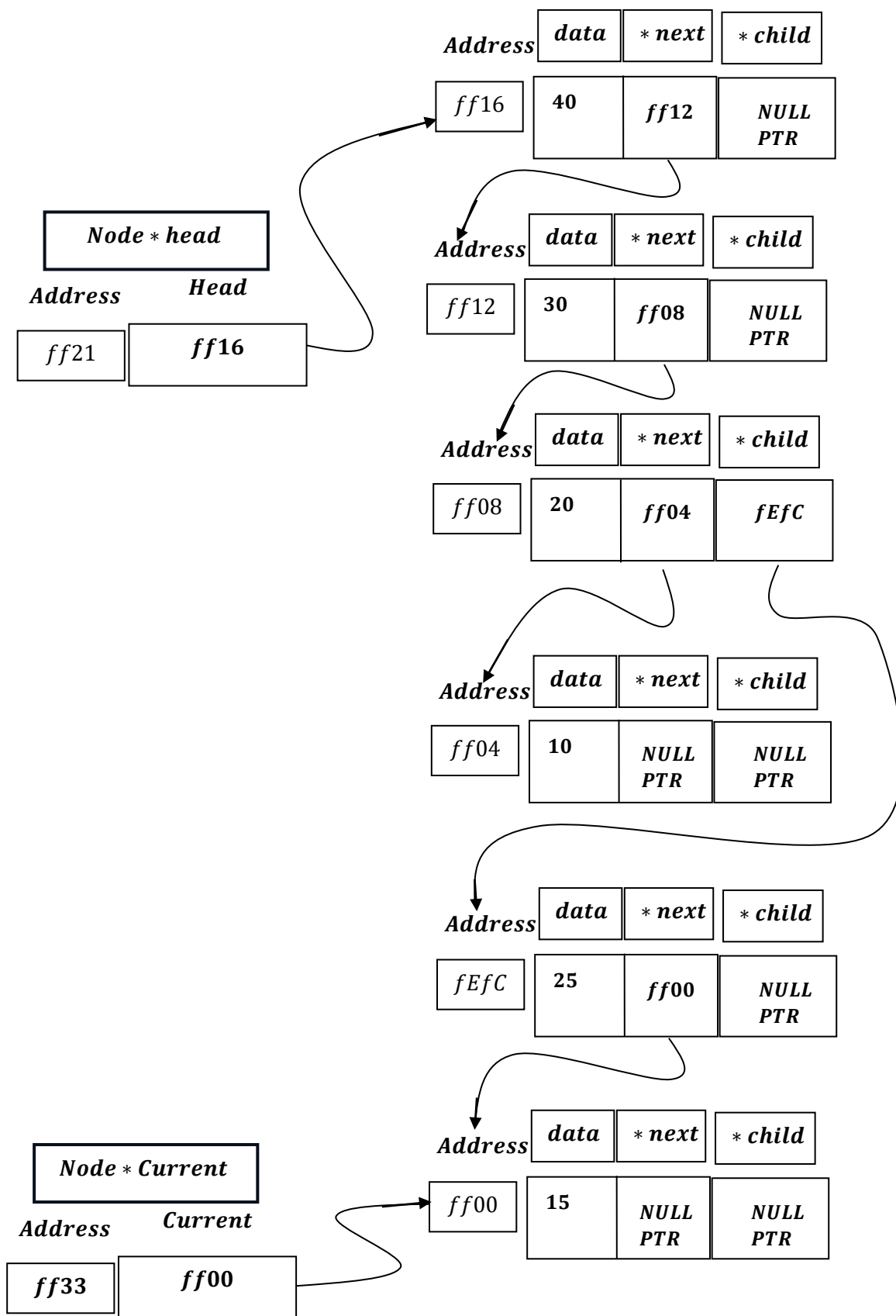
```
searchNode(current: fEfC → next: ff00, target: 10);
```

Recursion child's next push – 1



```
Node *searchNode(Node *current, int target)
```

```
Node * searchNode(Node * current = ff00 , target = 10)
```



```
if (current == nullptr)
    return nullptr;
```

current $\neq$ *nullptr* , hence condition is not true or false.

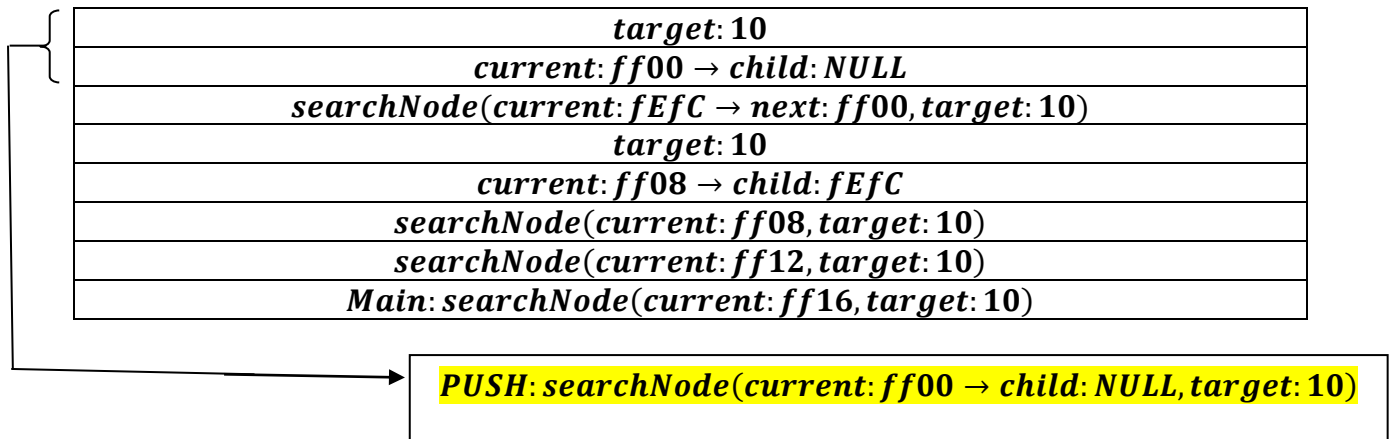
```
if (current->data == target)
    return current;
```

current $\rightarrow$ *data*: 15 $\neq$ *target* : 10 ,
 , hence condition is not true or false.

```
Node *foundInChild = searchNode(current->child, target);
```

searchNode(*current*: *ff00* $\rightarrow$ *child*: *NULL*, *target*: 10)

Recursion child's child push – 1



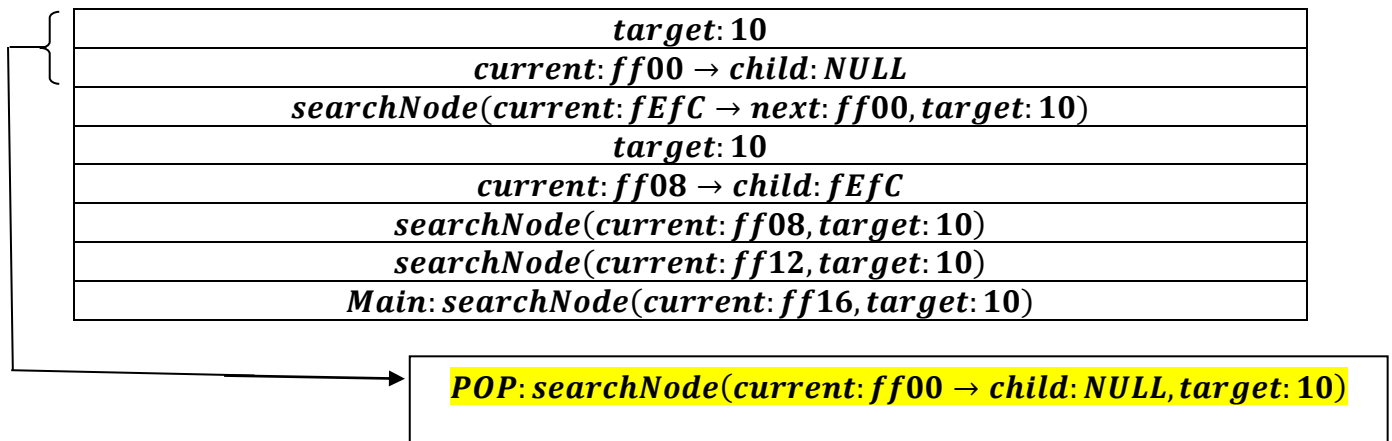
```
Node *searchNode(Node *current, int target)
```

Node * *searchNode*(*Node* * *current* = *NULL* , *target* = 10)

```
if (current == nullptr)
    return nullptr;
```

current = nullptr, hence condition is true, return nullptr.

Recursion child's child pop – 1



As soon as it pop, NULLPTR, now :

```
Node *foundInChild = searchNode(current->child, target);
```

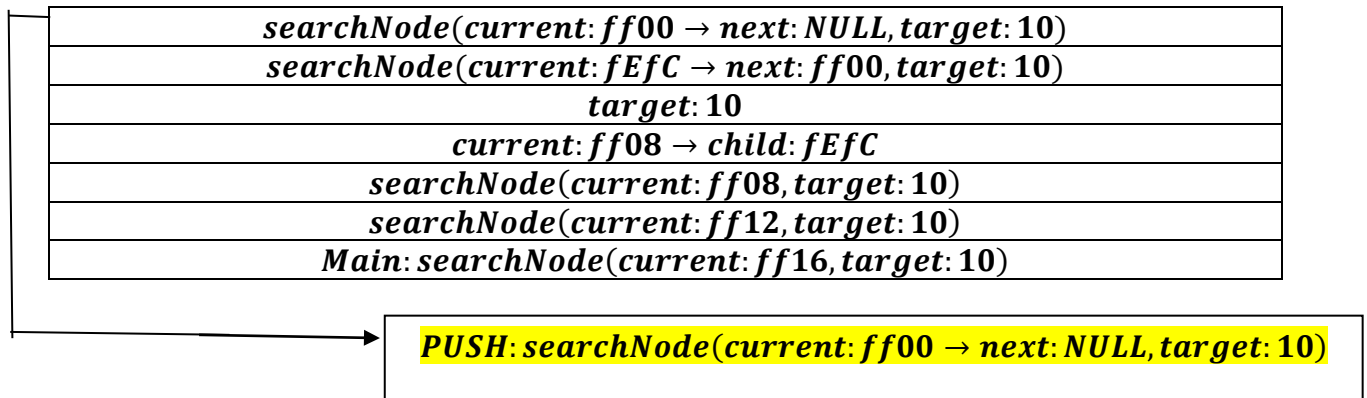
Node * foundInChild = NULLPTR.

```
if (foundInChild != nullptr)
{
    return foundInChild;
}
```

*But **foundInChild = NULLPTR;** hence condition is false.*

```
return searchNode(current->next, target);
searchNode(current: ff00 → next: NULL, target: 10);
```

Recursion child's next push – 2



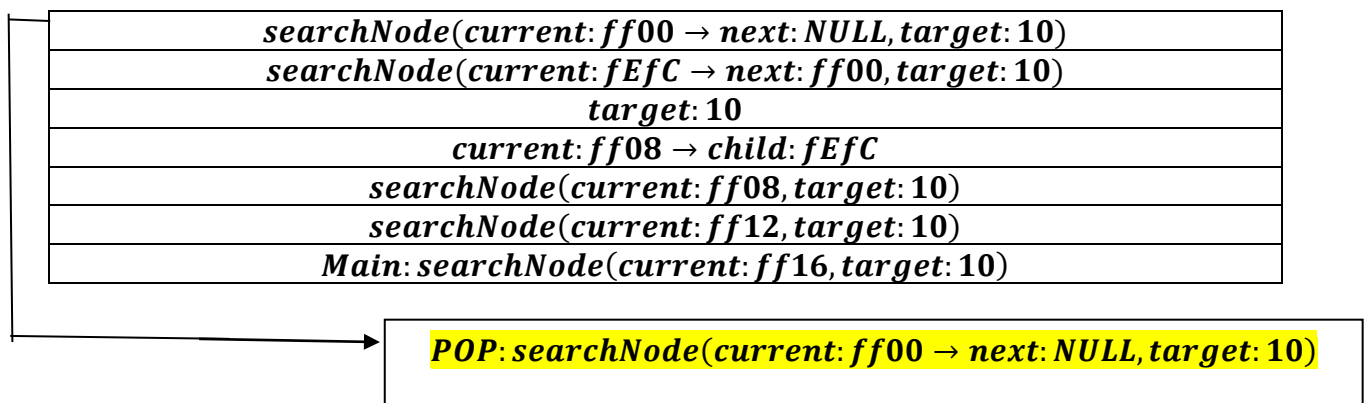
```
Node *searchNode(Node *current, int target)
```

```
Node * searchNode(Node * current = NULL ,target = 10)
```

```
if (current == nullptr)
    return nullptr;
```

current = nullptr , hence condition is true, return nullptr.

Recursion child's next pop – 2



As here return statement:

```
return searchNode(current->next, target);
```

∴ During POP operation :

POP: searchNode(current: ff00 → next: NULL, target: 10)

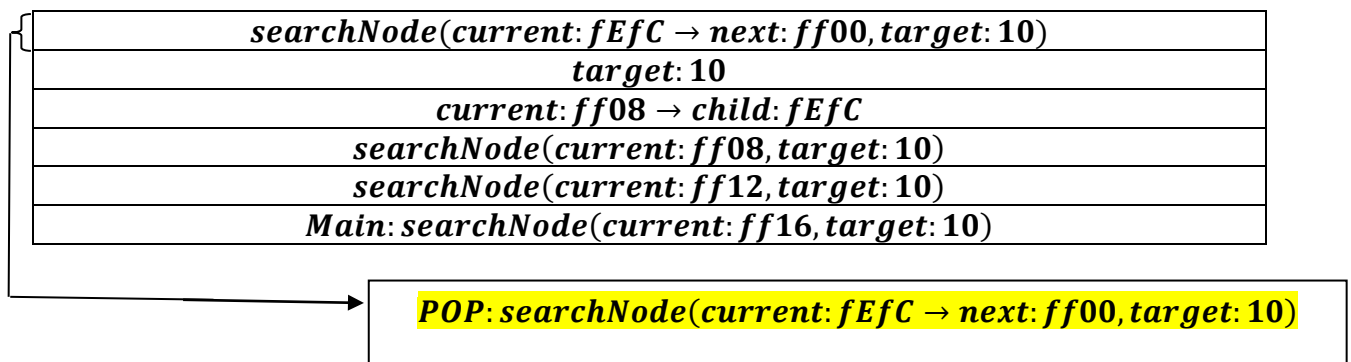
It returns: NULLPTR.

Now, as both the , ff00 → next = NULL and ff00 → child = NULL , completed and returns nullptr , this nullptr is handed over to :

searchNode(current: fEfC → next: ff00, target: 10)

And, searchNode(current: fEfC → next: ff00, target: 10) also return NULLPTR and get popped.

Recursion child's next pop – 3



As here return statement:

```
return searchNode(current->next, target);
```

∴ During POP operation :

POP: searchNode(current: fEfC → next: ff00, target: 10)

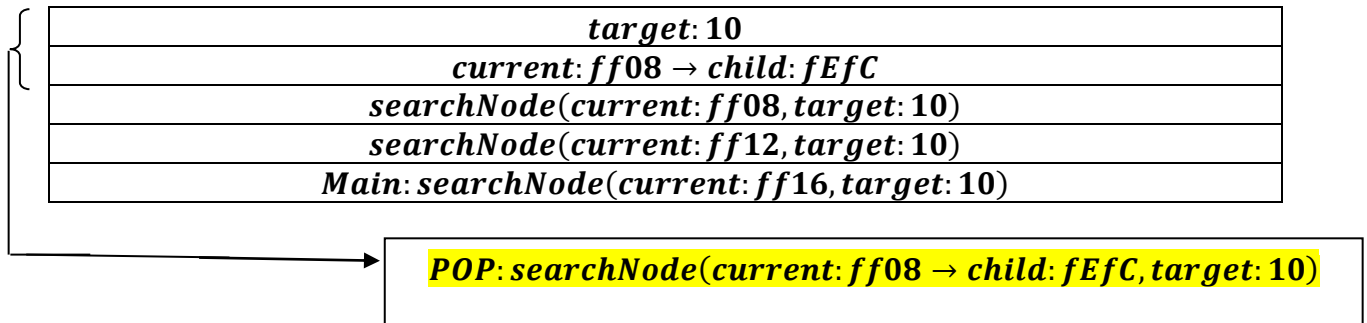
It returns: NULLPTR.

*Now, we are on frame : searchNode(current: ff08 → child: fEfC, taget : 10)
which was called and in operation, assigned to NULLPTR as,*

POP: searchNode(current: fEfC → next: ff00, target: 10) returns NULLPTR.

*As, searchNode(current: ff08 → child: fEfC, taget : 10) = NULLPTR, it also
gets popped, as this means the stack frame
searchNode(current: ff08 → child: fEfC, taget : 10) is finished.*

Recursion next's child pop



*Now, searchNode(current: ff08, target: 10) which was paused suddenly resumes
and realizes :My child branch was a dead end, so now, let's execute the next line of codes.*

As, searchNode(current: ff08 → child: fEfC, taget : 10) = NULLPTR

```
Node *foundInChild = searchNode(current->child, target);
```

∴ Node * foundInChild = NULLPTR.


```

if (foundInChild != nullptr)
{
    return foundInChild;
}

```

But ***foundInChild = NULLPTR***; hence condition is false.

```

return searchNode(current->next, target);
searchNode(current: ff08 → next: ff04, target: 10);

```

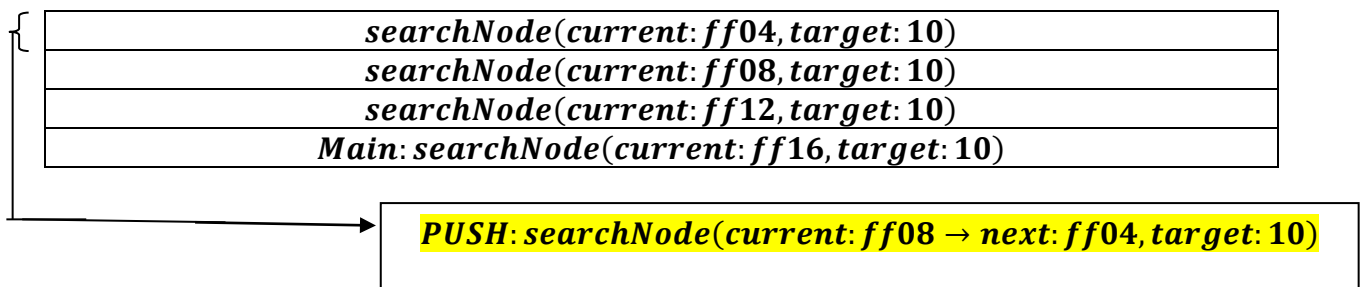
```

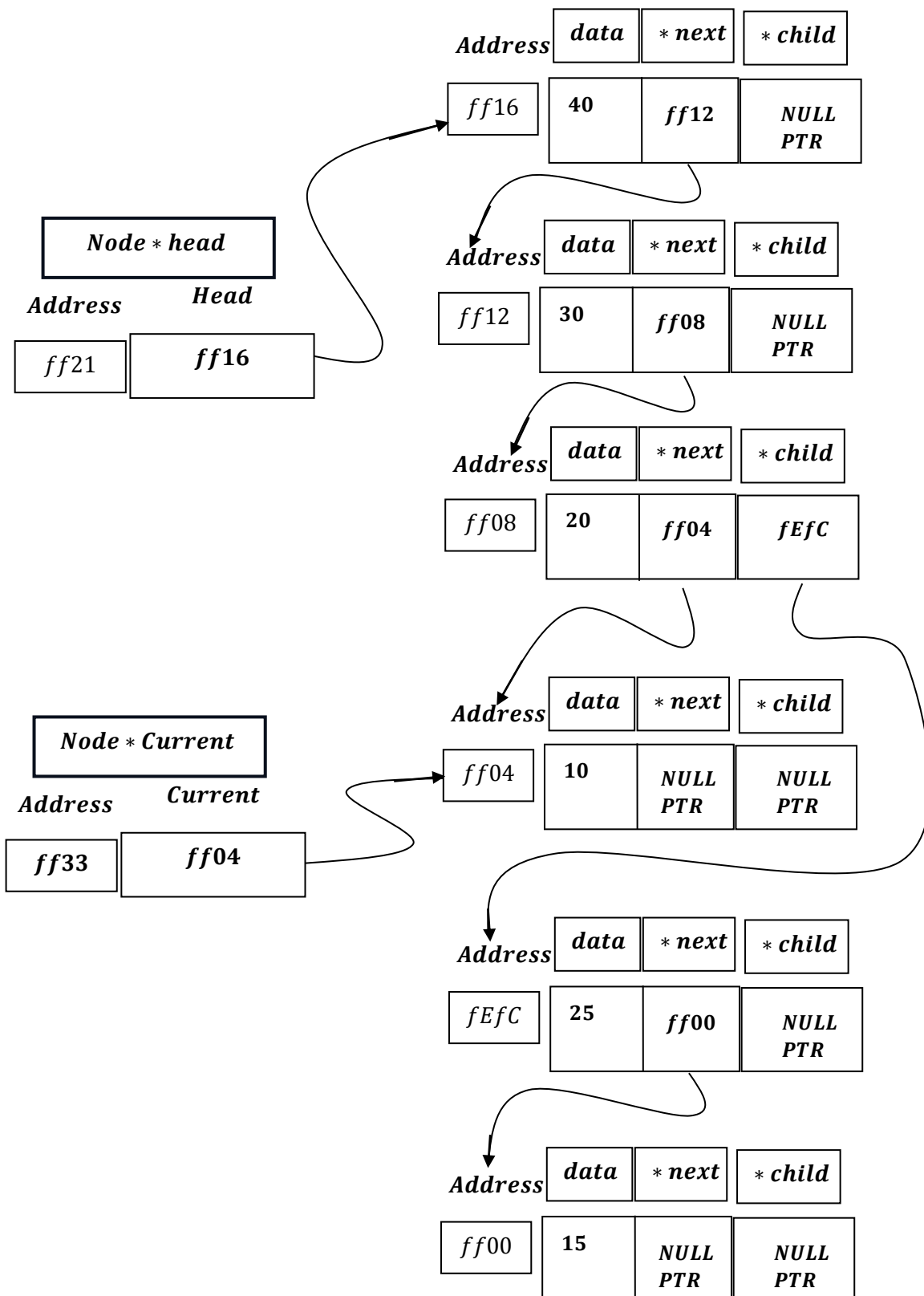
Node *searchNode(Node *current, int target)

```

*Node * searchNode(Node * current = ff04 , target = 10)*

Recursion next's push





```
if (current == nullptr)
    return nullptr;
```

current $\neq$ *NULLPTR*, hence condition is false.

```
if (current->data == target)
    return current;
```

current $\rightarrow$ *data*: 10 = *target*: 10, condition matches hence return
current : ff04.

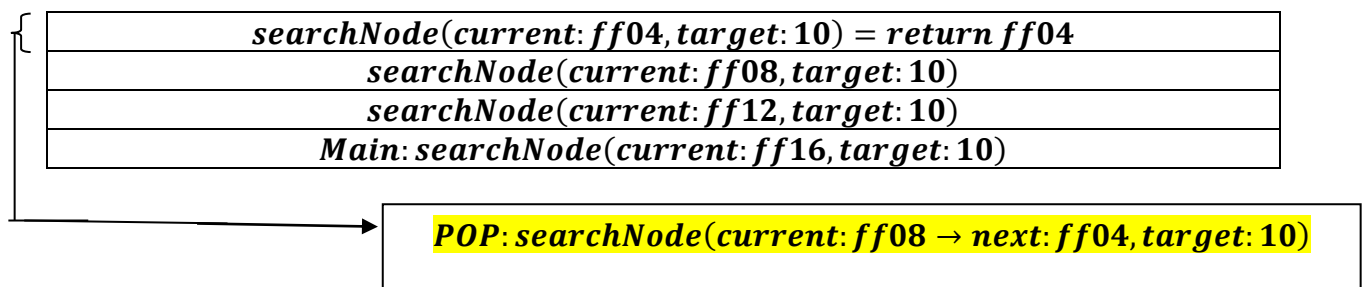
As soon as *current* return : ff04. The recursive call stacks starts
unwinding:

searchNode(current: ff04, target: 10) returns ff04 due to :

```
return searchNode(current->next, target);
```

return statement.

Recursion next's pop (Unwinding) – 1



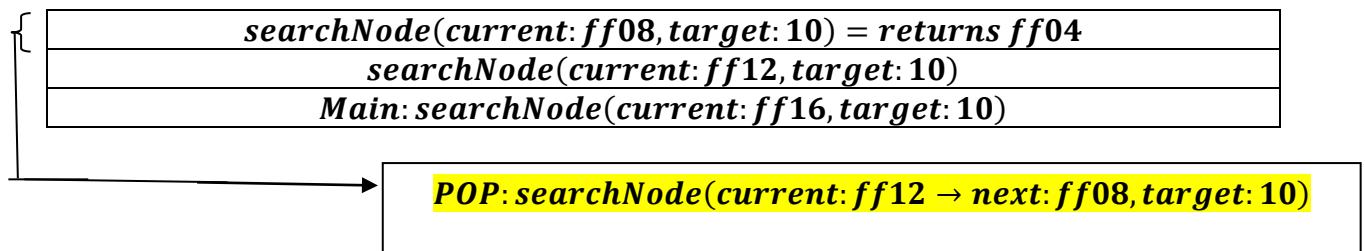
searchNode(current: ff04, target: 10) returns ff04 passes to call :

searchNode(current: ff08, target: 10) , and due to :

```
return searchNode(current->next, target);
```

return statement, it also returns ff04.

Recursion next's pop (Unwinding) – 2



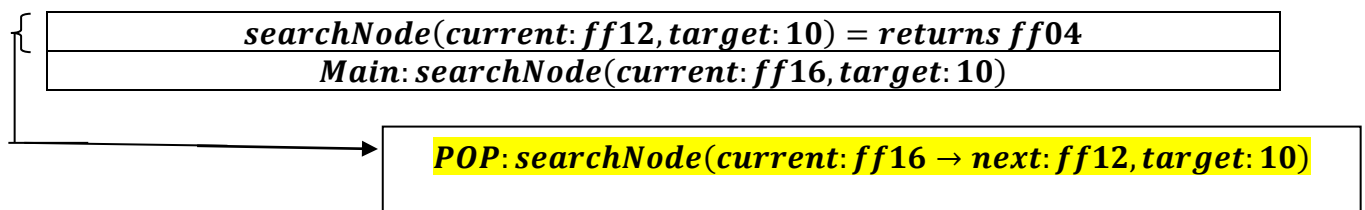
searchNode(current: ff08, target: 10) returns ff04 passes to call :

searchNode(current: ff12, target: 10) , and due to :

```
return searchNode(current->next, target);
```

return statement, it also returns ff04.

Recursion next's pop (Unwinding) – 2



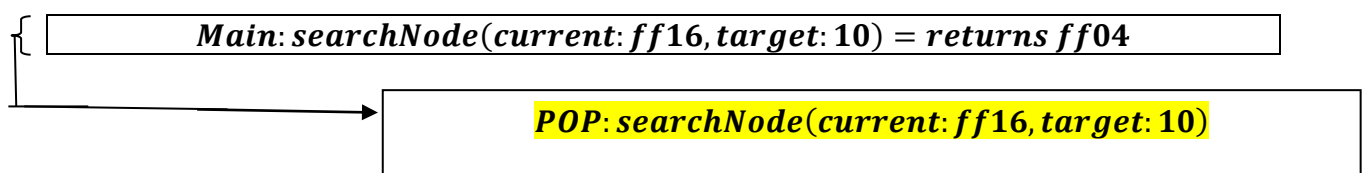
searchNode(current: ff12, target: 10) returns ff04 passes to call :

searchNode(current: ff16, target: 10) , and due to :

```
return searchNode(current->next, target);
```

return statement, it also returns ff04.

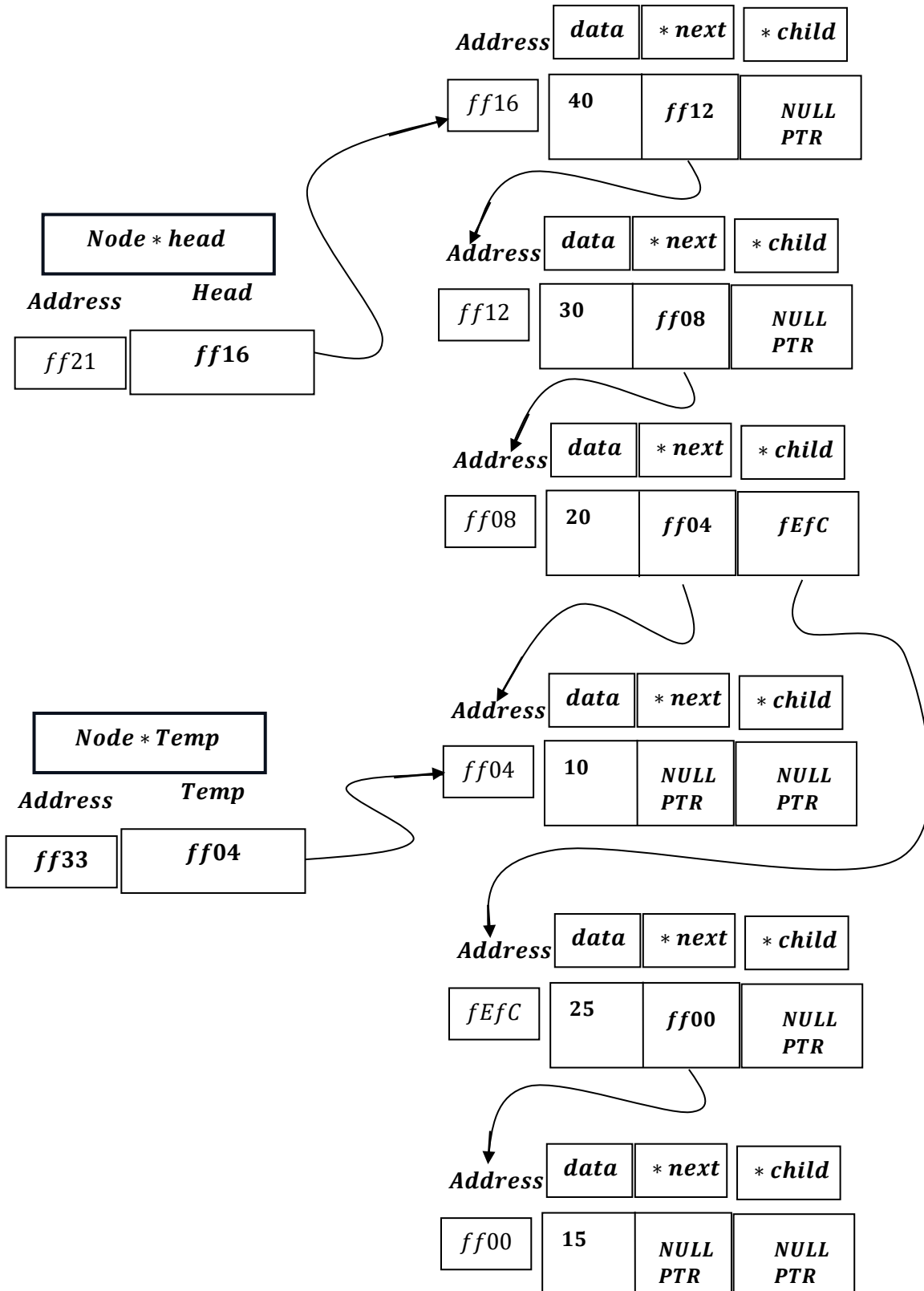
Recursion next's pop (Unwinding) – 2



Hence return : *ff04*.

```
Node *temp = searchNode(head, target);
```

*Node * temp = ff04.*



Main List's Child Node Insertion

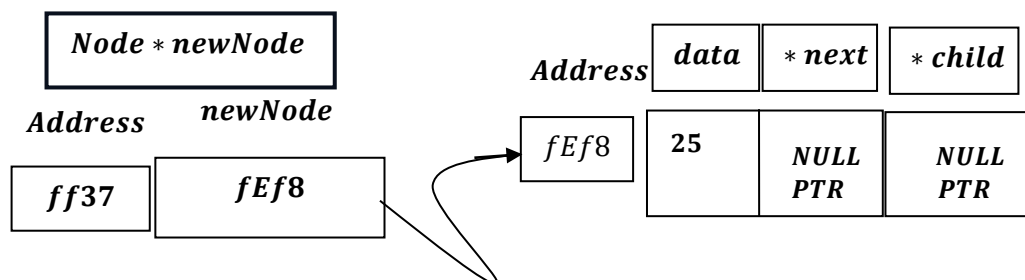
```
if (temp == nullptr)
{
    cout << "Element " << target << " not found anywhere
in the list." << endl;
    return;
}
```

Temp : ff00 $\neq$ NULLPTR, hence condition is false.

```
cout << "Enter data to insert: ";
cin >> data;
Node *newNode = createNode(data);
```

Enter data : 5.

```
Node *newNode = createNode(data);
```



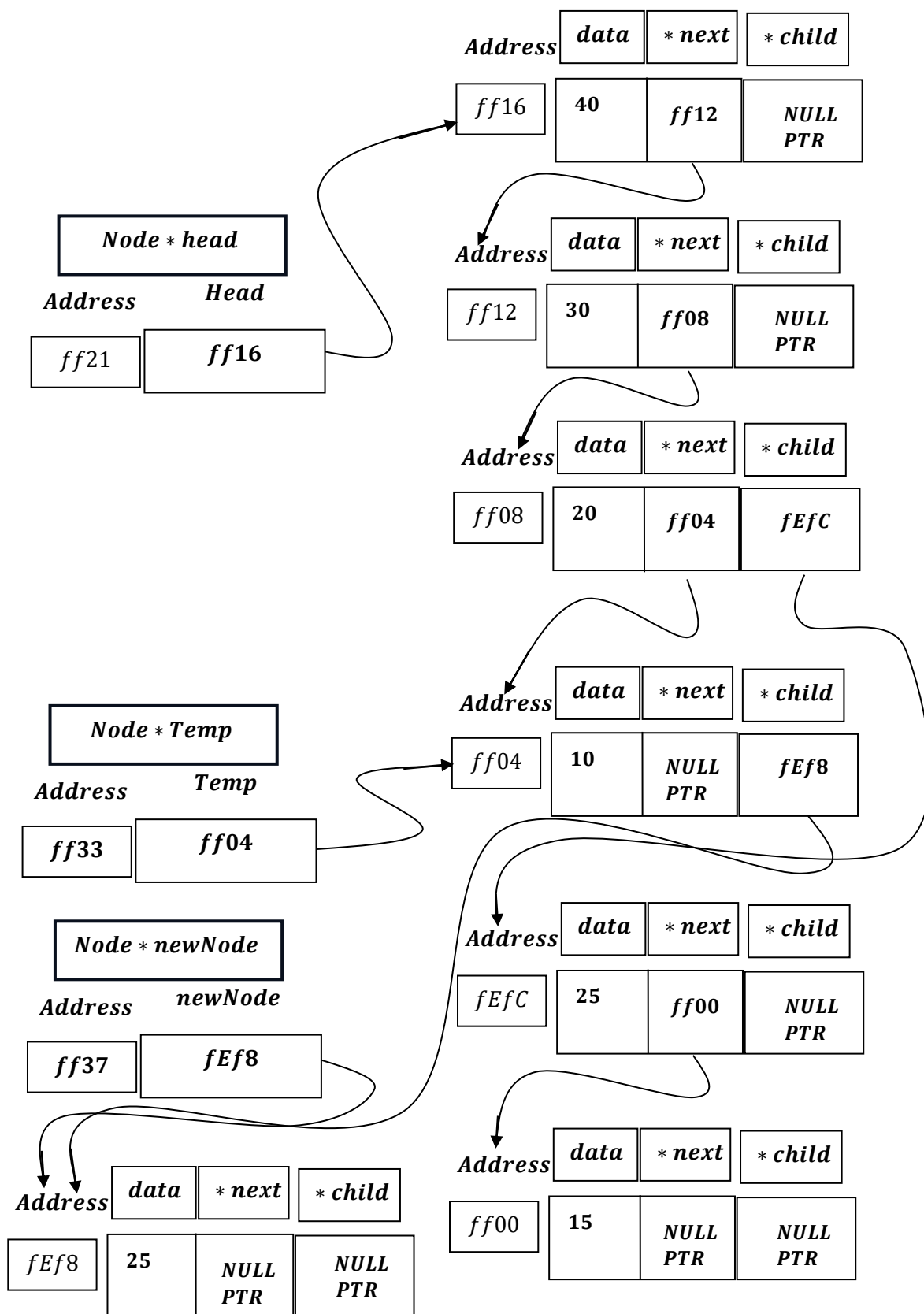
```
cout << "Insert as (1) Next Node or (2) Child Node? ";  
cin >> type;
```

Lets say type: 2 (Child)

```
else if (type == 2)  
{  
    newNode->next = temp->child;  
    temp->child = newNode;  
    cout << "Inserted " << data << " as CHILD of " <<  
target << endl;  
}
```

newNode → next = temp → child = NULLPTR

Temp → child = newNode = fEf8



And after function end we have:

